

# From Domain Knowledge to Composable Reactive Code: LLM-Assisted Development with Scenario-Based Programming

Adiel Ashrov<sup>1</sup>, Tom Yaacov<sup>2</sup>, and Guy Katz<sup>1</sup>

<sup>1</sup> The Hebrew University of Jerusalem, Jerusalem, Israel  
{adiel.ashrov,g.katz}@mail.huji.ac.il

<sup>2</sup> King's College London, London, United Kingdom  
{tom.yaacov}@kcl.ac.uk

**Abstract.** Reactive systems must continuously respond to events and coordinate interacting behaviors, making code generation challenging: reliability depends not only on individual components, but also on how they compose at runtime. We investigate *Scenario-Based Programming (SBP)* in two complementary roles: as a structured medium for communicating domain knowledge to a *Large Language Model (LLM)*, and as the target paradigm for generated extensions. We implement the *Taki* card game in SBP and ask an LLM to generate Python players using progressively richer contexts containing domain knowledge cast through SBP. Separately, we ask the LLM to generate strategy scenarios that compose directly with the existing implementation. In controlled simulations, all evaluated generated strategies complete all simulation runs without observed runtime errors, deadlocks, or draws. The generated SBP strategy also composes with an additional hand-authored scenario without observed execution failures. Richer SBP-cast domain knowledge is associated with stronger generated Python strategies: their win rate against a fixed random SBP baseline increases from 59.4% to 62.5%. The directly generated SBP strategy wins 64.9% against the same baseline and 51.2%–52.3% in head-to-head comparisons with the generated Python strategies. These results suggest that SBP can serve both as a medium for communicating domain knowledge and as a composable target for LLM-generated reactive-system extensions.

**Keywords:** Scenario-Based Programming · AI for Software Engineering · Large Language Models · Reactive Systems · LLM-Based Code Generation.

## 1 Introduction

*Large Language Models (LLMs)* increasingly assist software developers with code-generation tasks [8, 13, 21, 28, 30]. However, generating reliable code for complex reactive systems remains challenging [7, 21]. In such systems, correctness depends not only on the behavior of individual components in isolation, but also on how they coordinate over time. A generated component may therefore appear correct in isolation while violating system-level assumptions when

integrated with existing code, for example, by enabling invalid event sequences, blocking required actions, or disrupting the coordination among existing behaviors. The challenge is thus not only to generate locally correct code, but also to preserve the coordination constraints of the surrounding system. This motivates development approaches that expose these constraints to the LLM in a form that supports compositional extension.

One promising approach to addressing this challenge is *Scenario-Based Programming (SBP)* [17], which is a generalization of *Live Sequence Charts (LSC)* [9]. In SBP, system behavior emerges from independent *b-threads*, each typically capturing a single concern, which a central execution engine weaves together by selecting requested events that are not blocked. This architecture supports modularity, since each b-thread expresses a requirement independently, and composability, since new concerns can often be addressed by introducing additional b-threads without modifying existing ones. Prior work suggests that this decomposition aligns with how people naturally describe system requirements [11], motivating the hypothesis that SBP may ease both LLM-assisted implementation and the communication of domain knowledge to an LLM through a prompt. Despite these apparent affinities, the synergy between SBP and LLM-assisted code generation has received limited empirical evaluation, particularly for incremental extensions to an existing reactive system.

In this paper, we investigate the combination of SBP and LLM-assisted code generation in two complementary roles. We study the use of SBP as a structured representation through which domain knowledge is communicated to the LLM, and as the target paradigm in which generated extensions are implemented. As a testbed, we implement *Taki* [27]—a complex card game involving action cards, multi-step sequences, and asymmetric player information—using BPPy [29]. We ask a target LLM to generate Python players under progressively richer contexts containing domain knowledge cast through SBP, and separately to generate strategy b-threads that compose directly with the existing SBP implementation. We evaluate the resulting strategies through repeated controlled simulations. Specifically, we study whether the generated Python and SBP strategies execute and integrate without observed failures (**RQ1**), whether the generated SBP strategy achieves competitive strategic performance relative to the generated Python players (**RQ2**), and whether richer domain knowledge cast through SBP improves LLM-generated Python players and, when cast as strategy b-threads, improves performance over a random baseline (**RQ3**).

We evaluate the generated strategies through 10,000-game pairwise simulations with balanced starting positions and a fixed seed schedule, reporting overall and position-conditioned win rates together with Wilson score confidence intervals. All evaluated Python and SBP strategies complete their simulation runs without observed runtime errors, deadlocks, or draws. The generated SBP strategy also composes with an additional hand-authored b-thread without observed execution failures (**RQ1**). The directly generated SBP strategy remains competitive in head-to-head comparisons with the generated Python strategies, winning between 51.2% and 52.3% of games (**RQ2**). Providing the LLM with richer domain knowledge cast through SBP is associated with stronger generated Python

strategies: their win rate against the same random SBP baseline increases from 59.4% for the base configuration to 62.5% when the context also includes example strategy b-threads. The directly generated SBP strategy likewise outperforms this baseline, winning 64.9% of games (**RQ3**). These results suggest that SBP can serve both as a structured medium for communicating domain knowledge to an LLM and as a composable target for generated reactive-system extensions.

The rest of the paper is organized as follows. Section 2 introduces SBP and the BPy framework. Section 3 presents the Taki testbed, its SBP implementation, and the hand-authored strategy refinements used in the study. Section 4 describes the research questions, strategy-generation pipeline, experimental configurations, evaluation criteria, and simulation protocol. Section 5 reports the experimental results, and Section 6 discusses their implications and threats to validity. Section 7 reviews related work, and Section 8 concludes and outlines directions for future work.

## 2 Scenario-Based Programming and BPy

We begin by introducing the key concepts of scenario-based programming (SBP). For a grounded discussion, we illustrate these concepts using the BPy package [29], a framework for SBP in Python.

In SBP, system behavior is specified through scenarios named *b-threads*. These are sequential threads of execution, each specifying behavior that the system should include or avoid. B-threads are usually independent and self-contained, with each one focusing on a particular aspect of the system, often corresponding to an individual requirement. At runtime, an application-agnostic execution mechanism coordinates the b-threads and combines their individual specifications into a unified system behavior. This coordination follows the synchronization protocol introduced by Harel, Marron, and Weiss in [16]. Before every event generated by the b-program is selected, each b-thread submits a synchronization statement declaring which events it *requests*, which events it *waits for* (without actively requesting), and which events it *blocks* from occurring. After submitting its statement, the b-thread is paused. When all b-threads have submitted their statements, the b-program reaches a *synchronization point*. An event arbiter then selects one event that is requested by at least one b-thread and is not blocked by any b-thread. All b-threads that requested or waited for the selected event are resumed, while the remaining b-threads stay paused and retain their statements for the next synchronization point. This synchronization and event-selection cycle continues throughout the execution of the b-program.

In addition to request, waitFor, and block, the BPy strategy used in this work associates requested events with numeric priorities. Lower values denote stronger selection preference. At each synchronization point, the arbiter removes blocked events, keeps the requested events with the lowest priority value, and selects uniformly at random among ties. We use this mechanism to encode preferences among simultaneously available actions, as described in Sections 3.1 and 4.5. The formal semantics of SBP are available in [2].

To illustrate these concepts, we begin with a simple example b-program implemented using BPPy. The example adapts a sample b-program introduced by Harel et al. [17], in which a system controls the mixing of hot and cold water from two separate taps. Listing 1.1 presents two b-threads, `add_hot` and `add_cold`, each of which requests three events representing the pouring of a small amount of hot or cold water. In BPPy, b-threads are implemented as Python generators, i.e., functions that can suspend their execution and return data to the caller using the `yield` idiom, and later resume execution when invoked through the `send` method. Synchronization statements are represented by instances of the `sync` class, which specify events or sets of events through the `request`, `block`, and `waitFor` arguments. The BPPy execution mechanism initially invokes each b-thread generator independently and waits for it to yield a synchronization statement. After collecting statements from all b-threads, the mechanism selects an event and continues the execution of the b-program according to the synchronization protocol described above.

```
@bp.thread
def add_hot():
    for i in range(3):
        yield bp.sync(request=bp.BEvent("HOT"))

@bp.thread
def add_cold():
    for i in range(3):
        yield bp.sync(request=bp.BEvent("COLD"))
```

Listing 1.1: The `add_hot` and `add_cold` b-threads. Each requests its corresponding event three times.

In contrast to many programming paradigms, SBP does not require developers to define a single predetermined system behavior. Rather, the system may follow any behavior that is consistent with the constraints imposed by all specified b-threads. For example, a b-program containing only the two b-threads in Listing 1.1 places no restrictions on the order in which hot and cold water are poured. Its execution may produce any sequence containing exactly three occurrences of the `HOT` event and three occurrences of the `COLD` event, such as `COLD, HOT, HOT, COLD, HOT, COLD` or `COLD, COLD, COLD, HOT, HOT, HOT`.

To further illustrate, suppose that, after the initial system has been in use for some time, a safety issue is identified and a new requirement is introduced: two `HOT` events should not occur consecutively. One possible solution is to modify the existing `add_hot` b-thread by incorporating additional conditions and synchronization statements. However, SBP encourages maintaining the alignment between individual b-threads and the requirements they represent by introducing a separate b-thread for the new requirement. This supports an incremental and modular development process, in which behaviors can be added or removed independently without modifying existing b-threads. Accordingly, Listing 1.2 introduces the `control` b-thread, which repeatedly waits for a `HOT` event and, once such an event occurs, blocks the next `HOT` event while waiting for any subsequent event through the `All` event set.

Although this paper focuses primarily on BPPy, we note that SBP has also been implemented in a variety of languages, including Live Sequence Charts

(LSC) [9, 14], JavaScript [6], Java [16], and C++ [12]. The concepts presented in this work may therefore be applicable to these implementations as well. Beyond its different implementations, SBP has been employed in a diverse range of application domains. Examples include the development of a reactive IoT building [10], a fully operational nano-satellite [4], an autonomous rover [19], and a model-driven testing tool suite [5].

```
@bp.thread
def control():
    while True:
        yield bp.sync(waitFor=bp.BEvent("HOT"))
        yield bp.sync(waitFor=bp.All(), block=bp.BEvent("HOT"))
```

Listing 1.2: The `control` b-thread, which prevents the `HOT` event from occurring consecutively.

### 3 TAKI as a Complex Reactive System

*Taki* is a shedding-type card game in which the first player to discard all cards from their hand wins [27]. At the beginning of a game, each player receives eight cards, the remaining cards form the *draw pile*, and one card is placed face-up to start the *discard pile*. The top card of this pile, referred to as the *leading card*, determines which cards may be played next. On each turn, the active player may discard a card that matches the color, number, or type of the leading card. Certain action cards may also be played according to their own rules. If the player has no legal card, the player draws one card and the turn passes to the next player. Thus, even before considering action cards, the set of legal moves changes after every discarded card and depends on both the current leading card and the active player’s hand. A visual overview of the gameplay is available in the accompanying video tutorial [26].

The official deck contains numbered cards in several colors together with action cards that modify the normal turn structure. We implement a two-player variant restricted to numbered cards and four action-card types. A *Stop* card skips the opponent’s next turn; consequently, in the two-player setting, the player who uses it immediately plays again. A *TAKI* card opens a sequence in which the active player may discard multiple cards of the *TAKI* card’s color during the same turn, until the player explicitly closes the sequence. A *Super TAKI* card is a wild *TAKI* card that may be played independently of the leading card and opens a sequence by inheriting the current leading color. A *change-color* card may also be played independently of the leading color and allows the active player to choose the color that constrains subsequent play. These cards introduce state that persists beyond the single action that creates it: the system must track whether a *TAKI* sequence is open, which color governs that sequence, whether the opponent’s turn is skipped, and which color is selected after a *change-color* card. Figure 1 illustrates the ordinary matching rule and the legal action cards in a representative game state. The remaining action cards defined in the official ruleset, as well as multi-player play, are outside the scope of this implementation.



Fig. 1: Illustration of a two-player Taki game state, with card (7, BLUE) as the leading card in the discard pile. If Player 1 is active, the legal moves are (1, BLUE) and (TAKI, BLUE), which match its color, and (7, RED), which matches its number; (5, GREEN) is not playable. If Player 2 is active, the legal moves are (STOP, BLUE), which matches the leading color, and the wild CHANGE-COLOR and SUPER TAKI cards; (3, RED) is not playable.

### 3.1 The SBP Taki Implementation

We implement the two-player Taki game as a b-program whose b-threads separate action generation, rule enforcement, and strategic preference. One instance of `player_behavior` is created for each player and maintains that player’s private hand. These b-threads request the actions currently available from their local perspective, including playing each held card, drawing a card, selecting a color, and closing a TAKI sequence. Separate rule-enforcement b-threads constrain these requests. In particular, `enforce_turns` blocks actions of the non-active player, whereas `enforce_card_placement` blocks card plays that violate the current placement constraint. Additional b-threads enforce card placement, manage dealing, game termination, and execution monitoring. The complete executable implementation, including omitted helper methods and initialization protocols, is available in the project repository [2].

The b-threads coordinate through player-specific events such as `P0:Play(card)`, `P0:DrawCard`, and `P0:ClosedTaki`, together with global control events such as `NextTurn` and `DonePostAction`. Each requested event carries a numeric priority, where a lower value denotes a stronger selection preference. At each synchronization point, the event-selection mechanism first collects the events requested by all b-threads and removes every event blocked by at least one b-thread. It then retains only requests having the lowest priority value among the remaining events and selects uniformly at random among them. Strategy b-threads may re-request selected actions with stronger priorities, thereby biasing the arbiter toward preferred legal moves without modifying the rule-enforcement b-threads. Also, the implementation separates action generation, rule enforcement, and strategic preference into independent b-threads that communicate only through the shared event-selection mechanism.

Listing 1.3 presents the b-thread that exposes one player’s possible actions to the event-selection mechanism. The b-thread maintains only the player’s local hand; it does not determine whether a requested card is currently legal or whether it is the player’s turn. Instead, it requests one `Play(card)` event for each held card and relies on independent rule b-threads to block inadmissible actions.

This separation allows the same player behavior to operate with different rule and strategy b-threads.

```
@bp.thread
def player_behavior(index, initial_hand):
    """Maintain one player's hand and request their available actions."""
    hand = list(initial_hand)
    draw_card = player_event(index, "DrawCard", priority=20.0)
    deal_card = player_event(index, "DealCard", priority=10.0)
    no_more_cards = player_event(index, "NoMoreCards", priority=8.0)
    while True:
        req_events = [play_event(index, card, priority=10.0) for card in hand]
        req_events.append(draw_card)
        event = yield bp.sync(request=req_events)
        if event.name == draw_card.name:
            dealt_event = yield bp.sync(waitFor=deal_card)
            hand.append(extract_card_from_event(dealt_event))
        else:
            card = extract_card_from_event(event)
            hand.remove(card)
            handle_card(card)
        if not hand:
            yield bp.sync(request=no_more_cards)
            return
        yield bp.sync(request=BPEvent("NextTurn", priority=10.0))
```

Listing 1.3: Simplified `player_behavior` b-thread. For one indexed player, the b-thread maintains the player's hand and requests all candidate card-play actions together with a lower-priority draw action. It updates the hand after played or dealt cards, handles TAKI and change-color continuations in the `handle_card` function, and signals either turn completion or an empty hand. Independent b-threads enforce turn order and card legality and may re-request selected actions with stronger strategic priorities.

Listing 1.4 presents the turn-order b-thread, which tracks the active player, blocks the opponent's actions, and handles the special control flow introduced by Stop and TAKI-family cards.

```
@bp.thread
def enforce_turns(starting_player=0):
    curr_player = starting_player
    next_player = 1 - curr_player
    next_turn = BPEvent("NextTurn", priority=10.0)
    done_post_action = BPEvent("DonePostAction", priority=10.0)
    while True:
        event=yield bp.sync(waitFor=[next_turn, player_play_event_set(curr_player)],
                           block=other_player_action_events(curr_player))
        if event.name == next_turn.name:
            curr_player = next_player
            next_player = 1 - curr_player
            continue
        card = extract_card_from_event(event)
        if is_stop(card):
            next_player = curr_player
            yield bp.sync(request=done_post_action,
                           block=all_player_action_event_set())
        elif is_taki_family(card):
            closed_taki = player_event(curr_player, "ClosedTaki")
            yield bp.sync(waitFor=closed_taki,
                           block=other_player_action_events(curr_player))
            yield bp.sync(request=done_post_action,
```

```
block=all_player_action_event_set()
```

Listing 1.4: Simplified `enforce_turns` b-thread for the two-player variant. The b-thread tracks the active and next players, blocks actions of the non-active player, and transfers control when `NextTurn` occurs. A Stop card assigns the next turn to the same player, whereas a TAKI-family card keeps the opponent blocked until the active player’s `ClosedTaki` event occurs. The b-thread then requests `DonePostAction` before further player actions occur. The helper `player_play_event_set(i)` matches all card-play events of player  $i$ ; `other_player_action_events(i)` matches gameplay actions of the non-active player; and `all_player_action_event_set()` matches gameplay actions of both players. Their definitions are omitted for readability.

For validation, we implemented three monitoring b-threads that observe generated execution traces: one checks that every player action belongs to the active player, a second detects deadlock when no requested and unblocked event is available, and a third bounds the number of consecutive player events without a completed turn, recording the run as a draw when this bound is exceeded. These monitors are non-strategic and are also reused as assertions for BPPy’s explicit model-checking verifier (`DFSBProgramVerifier`) [29], which issues offline queries to check whether a turn-order violation, deadlock, or livelock is reachable independently of the simulated games.

### 3.2 Hand-Authored SBP Strategy Refinements

To illustrate how strategic knowledge can be added to the SBP Taki implementation, we implemented three hand-authored strategy refinements. These refinements are designed by the researchers as simple heuristics for improving the basic random SBP player. They are not claimed to be optimal Taki strategies; rather, they provide controlled examples of how strategy-level knowledge can be expressed as modular b-threads that compose with the rule-enforcement b-threads described above.

The first refinement, `TAKI-FAMILY-PREFERENCE`, assigns the same stronger selection preference to all TAKI-family cards, including both regular TAKI and Super TAKI. The motivation is that TAKI-family cards can open multi-card sequences, and therefore may allow the player to reduce their hand more quickly than ordinary single-card moves. The implementation of this strategy is available in the supplementary material [2].

The second refinement, `TAKI-OVER-SUPER-TAKI`, introduces a more fine-grained priority hierarchy among TAKI-family cards. Regular TAKI cards receive the strongest selection preference, Super TAKI cards receive an intermediate selection preference, and other cards keep their default priority. Thus, the strategy still prefers TAKI-family cards over ordinary cards, but also prefers opening a regular TAKI over Super TAKI when both are available. Listing 1.5 shows a simplified version of this strategy b-thread. The listing omits player indices, concrete event-name construction, and helper definitions for readability.

The third refinement, `CONSERVE-SUPER-TAKI`, combines `TAKI-OVER-SUPER-TAKI` with an additional b-thread that blocks Super TAKI while a

regular TAKI sequence is already open. The motivation is to conserve Super TAKI for later states in which its wild-card behavior may be more useful, instead of spending it inside a sequence that is already constrained by the color of a regular TAKI. The implementation of this strategy is available in [2].

```
@bp.thread
def prefer_taki_over_super_taki_strategy(index, initial_hand):
    hand = list(initial_hand)
    draw_card = player_event(index, "DrawCard", priority=20.0)
    deal_card = player_event(index, "DealCard", priority=10.0)
    no_more_cards = player_event(index, "NoMoreCards", priority=8.0)
    def strategic_play_event(card):
        if is_regular_taki(card):
            priority = 4.0
        elif is_super_taki(card):
            priority = 6.0
        else:
            priority = 10.0
        return play_event(index, card, priority=priority)
    while True:
        requested_events = [strategic_play_event(card) for card in hand]
        event = yield bp.sync(request=requested_events,
                              waitFor=[draw_card, no_more_cards])
        if event.name == no_more_cards.name: return
        if event.name == draw_card.name:
            dealt_event = yield bp.sync(waitFor=deal_card)
            hand.append(extract_card_from_event(dealt_event))
            continue
        card = extract_card_from_event(event)
        hand.remove(card)
        if is_taki_family(card):
            yield from observe_taki_sequence_and_update(index, hand)
        if is_action_card(card):
            yield bp.sync(waitFor=BPEvent("DonePostAction"))
```

Listing 1.5: The `prefer_taki_over_super_taki_strategy` b-thread for one indexed player. The b-thread tracks the player’s hand and re-requests regular TAKI card-play events with priority 4.0 and Super TAKI card-play events with priority 6.0, while other cards retain the default priority 10.0. It observes draw, deal, action-completion, and game-ending events to keep its local hand state aligned with `player_behavior`.

Importantly, all three refinements leave the rule-enforcement b-threads unchanged. The first two refinements influence event selection by assigning lower numeric priority values to preferred events, while the third adds a temporary blocking constraint. This illustrates the compositional style of SBP: strategic knowledge can be added as small behavioral modules that interact with the existing game implementation through the shared event-selection mechanism.

### 3.3 TAKI as a Stress Test for Composition

Taki combines several sources of reactive complexity, including action cards that modify the normal turn structure, multi-event interactions such as TAKI sequences, and asymmetric player information. In our implementation, these concerns are captured by independent b-threads which interact in order to yield a complex system behavior, making Taki a representative benchmark for compositional reactive systems.

Consequently, generated strategies must compose correctly with existing rule-enforcement b-threads rather than implement the game logic independently. This makes Taki a suitable benchmark for evaluating whether LLM-generated behavioral extensions integrate into an existing reactive system without observed execution failures, a question revisited in Section 5.

## 4 Experimental Setup

Having described the SBP Taki implementation, we now present the experimental setup used to study whether SBP supports LLM-assisted strategy generation for reactive systems. The setup evaluates whether LLM-generated strategies execute and integrate without failures, examines how domain knowledge cast through SBP affects the resulting strategies, and compares the strategic performance of LLM-generated SBP strategy b-threads against LLM-generated Python strategies. We evaluate the LLM-generated strategies, together with a set of hand-authored SBP baselines, through repeated game simulations under controlled conditions. The section is organized around three research questions, followed by a description of the experimental pipeline, prompt configurations, evaluation dimensions, baselines, and simulation protocol.

### 4.1 Research Questions

We organize our evaluation around three research questions:

- **RQ1 — Executability and Integration:** Can LLM-generated strategies, whether implemented as a Python player or as SBP strategy b-threads, be executed and integrated into the Taki simulation framework without observed execution failures?
- **RQ2 — Strategic Performance:** Can LLM-generated SBP strategy b-threads achieve competitive strategic performance compared with LLM-generated Python players under the same conditions?
- **RQ3 — Domain Knowledge:** Does richer domain knowledge, cast through SBP, improve LLM-generated Python players, and does the same knowledge, cast as SBP strategy b-threads, improve performance over the same random baseline?

### 4.2 Experimental Pipeline

All experiments follow the same pipeline, shown in Fig. 2. For each configuration, we first define the context by selecting the available source files and supporting documentation. We then manually write a prompt, which is refined with the assistance of *Claude Sonnet 4.6* [1]. Claude is used only for prompt refinement and is kept separate from the target LLM that generates the strategies. This separation ensures that the evaluated model does not author or adapt its own instructions and keeps the prompt-construction process independent of the strategy-generation process. The context and prompt are then provided as input

to *Codex* [24], using *GPT-5.5* [25] as the target LLM, which reads them and generates the strategy code. During generation, the target LLM may inspect the source files, run tests, observe runtime errors, and revise its code, but it receives no additional human guidance, iterative prompting, or corrective feedback. The resulting strategy is then validated and evaluated according to the criteria described in Section 4.4. The pipeline remains fixed across configurations; only the context and its corresponding prompt vary, thereby determining the target implementation form and the extent to which SBP-cast domain knowledge is available to the model.

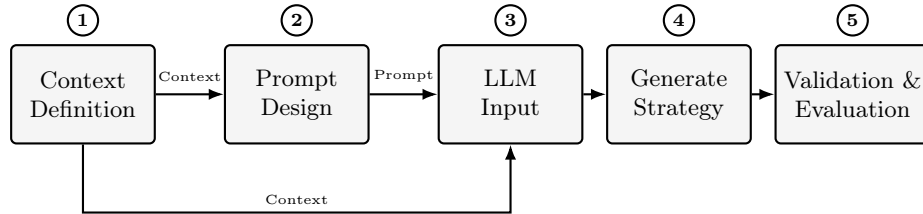


Fig. 2: The experimental pipeline used for strategy generation and evaluation.

### 4.3 Experimental Configurations

Table 1 summarizes the four experimental configurations. All configurations include the official Taki rulebook and the same high-level task: generate a strategy for one Taki player that selects or prioritizes player actions during gameplay in order to win. In the first three configurations, the generated artifact is a general Python agent that implements an interface method that maps a player-facing observation of the current game state to a chosen action. In the SBP-NATIVE configuration, the generated artifact is a set of strategy b-threads that compose with the existing behavioral program and influence event selection through the SBP synchronization mechanism.

The configurations are organized as an incremental progression: each configuration exposes the target LLM to additional implementation context or additional domain knowledge cast through SBP, relative to the previous one. In PY-BASE, the model receives only the official rulebook and the Python player interface described above, and is asked to generate a Python player that conforms to this interface. PY-SBP-CORE extends this setting by adding the BPy-based SBP Taki implementation, allowing the model to inspect how the game rules and event-level interactions are encoded in SBP. PY-SBP-STRATEGIES further extends the context with hand-authored SBP strategy b-threads introduced in Section 3.2, which serve as examples of how strategic domain knowledge can be expressed behaviorally, and are distinct from the generated Python players evaluated in this configuration. Finally, SBP-NATIVE changes the target artifact: the model receives the BPy-based SBP Taki implementation, the BPy API and programming idioms needed to write b-threads, and the same hand-authored SBP strategy examples, and is asked to generate new strategy b-threads that

compose directly with the existing behavioral program. We use *native* to convey that the generated artifact is expressed directly in SBP, in contrast to the first three configurations, whose generated artifact remains a Python player. Thus, the first three configurations evaluate the effect of progressively richer domain knowledge cast through SBP on Python strategy generation, while SBP-NATIVE evaluates direct generation of SBP strategy b-threads. All configurations and prompts are available in the project repository [3].

Table 1: Prompt configurations used in the strategy-generation experiments. The official rulebook is included in all configurations. Here, Py denotes Python-based strategy generation. Input strategy examples, when provided, are included as input context for the target LLM and are different from the generated strategies evaluated in the experiments. We use *as inspiration* when the examples are expressed in a different paradigm from the generated artifact, and *as reference* when the examples are expressed in the same paradigm as the generated artifact.

| Configuration     | Generated Artifact     | Code Context                                  | Input Strategy Examples                               |
|-------------------|------------------------|-----------------------------------------------|-------------------------------------------------------|
| PY-BASE           | Python player          | Python player interface                       | None                                                  |
| PY-SBP-CORE       | Python player          | Python player interface & BPPy-based SBP Taki | None                                                  |
| PY-SBP-STRATEGIES | Python player          | Python player interface & BPPy-based SBP Taki | Hand-authored SBP strategy b-threads (as inspiration) |
| SBP-NATIVE        | SBP strategy b-threads | BPPy-based SBP Taki & BPPy API                | Hand-authored SBP strategy b-threads (as reference)   |

#### 4.4 Evaluation Dimensions

We evaluate the generated strategies along three dimensions. We first assess executability and integration, then define the strategic-performance metrics used in the simulations, and finally use these metrics to study the effect of SBP-cast domain knowledge on Python strategy generation.

*RQ1 — Executability and Integration.* We assess whether each generated strategy can be loaded by the Taki simulation framework and participate in repeated games without observed execution failures. We record draws and deadlocks separately, as established in the simulation protocol (Section 4.6). In our experiments,

a draw denotes a run terminated by the livelock detector rather than a legitimate rule-based Taki outcome (Section 3.1). Such a run may indicate an integration or termination failure, but it is not necessarily a runtime error: unlike a crash, a livelock can also arise from a legal but non-terminating interaction between well-formed strategies. We therefore treat draws as RQ1-relevant warning signals rather than confirmed defects.

For SBP-NATIVE, we additionally evaluate compositional integration. We run self-play configurations in which the generated strategy b-threads are composed with and without an additional hand-authored b-thread, in both player positions. In these runs, we monitor for runtime errors, deadlocks, and draws. The results are reported in Section 5.2.

*RQ2 — Strategic Performance.* We evaluate strategic performance through repeated pairwise game simulations between the LLM-generated SBP-NATIVE strategy and each LLM-generated Python strategy. The primary metric is win rate, computed over the simulated games as described in Section 4.6. Since Taki exhibits a first-player advantage, we report both overall win rate and win rates conditioned on whether each strategy starts or plays second. We report Wilson score confidence intervals for the win rates and, where appropriate, use statistical tests to assess whether observed differences between strategies are significant.

*RQ3 — Domain Knowledge.* Using the strategic-performance metric defined above, we assess how progressively richer domain knowledge, cast through SBP, affects the generated strategies. For the Python configurations, we compare the win rates of PY-BASE, PY-SBP-CORE, and PY-SBP-STRATEGIES against the same SBP-RANDOM baseline. These three configurations keep the generated artifact fixed as a Python player, while exposing the target LLM to increasing amounts of domain knowledge cast through SBP. For SBP-NATIVE, only a single configuration is evaluated, so no corresponding richness progression can be constructed. We therefore evaluate SBP-NATIVE using the same baseline comparison — its win rate against SBP-RANDOM — but report it separately. Since SBP-NATIVE changes the generated artifact from a Python player to SBP strategy b-threads, in the context of RQ3, its result serves as a separate reference point rather than as another step in the Python progression.

#### 4.5 Baselines and Hand-Authored SBP Strategy Refinements

We use two families of baselines throughout our experiments: random strategies and hand-authored SBP strategy refinements.

*Random Baselines.* SBP-RANDOM and PYTHON-RANDOM implement the same random policy: each plays a legal card uniformly at random when one is available, and draws a card only when no legal card can be played. SBP-RANDOM uses the SBP Taki implementation without additional strategic refinements. In this implementation, player b-threads request available card actions with the same priority, while rule-enforcement b-threads block illegal actions. The draw

action is assigned a weaker selection preference under the priority mechanism described in Section 2, and is therefore selected only when no requested card action remains enabled. PYTHON-RANDOM implements the same policy directly through the Python player interface described in Section 4.3. Comparing these two random strategies directly (Section 5.1) serves as a validity control, indicating whether the two implementations produce comparable behavior under identical simulation conditions. SBP-RANDOM also serves as the fixed opponent for the RQ3 evaluations (Section 4.4).

*Hand-Authored SBP Strategy Refinements.* We use the three hand-authored SBP strategy refinements introduced in Section 3.2: TAKI-FAMILY-PREFERENCE, TAKI-OVER-SUPER-TAKI, and CONSERVE-SUPER-TAKI. Because these refinements are designed by the researchers rather than generated by an LLM, their results do not themselves constitute evidence for RQ1–RQ3. Instead, they serve two roles in the experimental setup. First, they provide researcher-designed heuristic baselines that extend the basic SBP-RANDOM player. Second, they are included as strategy examples in the PY-SBP-STRATEGIES (as inspiration) and SBP-NATIVE (as reference) prompt configurations, demonstrating how strategic knowledge can be expressed as small composable b-threads. The blocking b-thread used in CONSERVE-SUPER-TAKI is also reused in the composition test reported in Section 5.2.

#### 4.6 Simulation Protocol

All strategies are evaluated through repeated Taki simulations. For each comparison, we run 10,000 games using sequential random seeds starting from zero. For reproducibility, the seed schedule is fixed across experiments. To control for the observed first-player advantage, starting positions are balanced across each game sequence, so that each player starts approximately half of the games.

The experiments include three comparison groups. For RQ1, we monitor all runs for execution failures; for SBP-NATIVE, we also evaluate self-play settings, both with and without an additional hand-authored b-thread, to examine whether composition introduces observed conflicts or deadlocks. For RQ2, the LLM-generated SBP-NATIVE strategy is evaluated in direct head-to-head matchups against the LLM-generated Python strategies. For RQ3, each LLM-generated Python strategy is evaluated against the same fixed opponent, SBP-RANDOM, providing a common baseline for comparing the effect of progressively richer domain knowledge cast through SBP. The specific pairings, including additional comparisons with hand-authored SBP baselines, are reported in Section 5.

For each run, we record the winner, starting player, number of events, draw status, deadlock status, and runtime errors. A draw is recorded when the livelock detector triggers, as described in Section 3.1: no legal turn completes before a predefined player-event limit is reached. Win rates are computed over non-error runs. Draws and deadlocks are included in this denominator and reported separately; runtime errors are treated as incomplete runs and excluded from the win-rate statistics. Executability results are reported in Section 5.2.

## 5 Results

This section reports the empirical results of the study. We first establish the comparability of the random baselines and then examine the executability and integration of the generated strategies. Next, we evaluate how domain knowledge cast through SBP relates to strategy performance. Finally, we compare the directly generated SBP-NATIVE strategy with the generated Python strategies. Throughout this section, we report overall and position-conditioned win rates together with 95% Wilson score confidence intervals.

### 5.1 Random-Baseline Validity Control

We compare SBP-RANDOM and PY-RANDOM over 10,000 simulated games to assess whether the two implementations provide a suitable common reference point. As shown in Table 2, SBP-RANDOM wins 50.3% of the games, whereas PY-RANDOM wins 49.7%. A two-sided binomial test does not indicate a deviation from an even split ( $p \approx 0.58$ ), with no draws, deadlocks, or runtime errors across all games, averaging 93.5 events per game.

The starting player wins 52.1% of the games, and both strategies perform better when starting than when playing second. Thus, the comparison reveals a modest starting-position advantage but no evidence of a systematic performance advantage for either random implementation. We therefore use SBP-RANDOM as the common baseline in the subsequent evaluations.

Table 2: Comparison of SBP-RANDOM and PY-RANDOM over 10,000 games. CI denotes a 95% Wilson score confidence interval.

| Metric                       | SBP-RANDOM     | PY-RANDOM      |
|------------------------------|----------------|----------------|
| Overall win rate             | 50.3%          | 49.7%          |
| Win rate when starting       | 52.4%          | 51.8%          |
| Win rate when playing second | 48.2%          | 47.6%          |
| 95% CI when starting         | [51.0%, 53.8%] | [50.4%, 53.2%] |
| 95% CI when playing second   | [46.8%, 49.6%] | [46.2%, 49.0%] |

### 5.2 Executability and Integration

All evaluated generated strategies complete their simulation runs without observed runtime errors, deadlocks, or draws. This includes the three generated Python strategies and the directly generated SBP-NATIVE strategy. Thus, each generated artifact executes in its target environment and participates in repeated games without the execution failures monitored in this study.

We further evaluate whether SBP-NATIVE composes with an additional hand-authored b-thread that blocks Super TAKI during an open regular TAKI sequence. We test four self-play configurations, in which the additional b-thread

is assigned to neither player, either player individually, or both players. All 40,000 games complete without observed runtime errors, deadlocks, or draws. The full position-conditioned results are reported in the supplementary material [2].

Together, these results support an affirmative answer to RQ1: the generated strategies execute successfully, and SBP-NATIVE integrates with the tested behavioral extension. They do not constitute a general guarantee of correctness or conflict-free composition beyond the evaluated configurations.

### 5.3 Effect of Domain Knowledge Cast through SBP

We compare the three generated Python strategies against the same SBP-RANDOM baseline. The configurations provide the LLM with progressively richer domain knowledge cast through SBP: PY-BASE receives the game rules and Python player interface; PY-SBP-CORE additionally receives the SBP Taki implementation; and PY-SBP-STRATEGIES further receives example SBP strategy b-threads. Table 3 reports the results.

Table 3: Performance against SBP-RANDOM over 10,000 games each. See Sections 5.2 and 5.4 for the composition and head-to-head results, respectively.

| Strategy          | Overall | Starting | Second | 95% CI starting | 95% CI second  |
|-------------------|---------|----------|--------|-----------------|----------------|
| PY-BASE           | 59.4%   | 62.3%    | 56.5%  | [61.0%, 63.7%]  | [55.1%, 57.9%] |
| PY-SBP-CORE       | 59.8%   | 63.3%    | 56.4%  | [61.9%, 64.6%]  | [55.0%, 57.7%] |
| PY-SBP-STRATEGIES | 62.5%   | 65.7%    | 59.2%  | [64.4%, 67.0%]  | [57.9%, 60.6%] |
| SBP-NATIVE        | 64.9%   | 67.4%    | 62.4%  | [66.1%, 68.7%]  | [61.0%, 63.7%] |

All three generated Python strategies outperform the random baseline, with overall win rates ranging from 59.4% to 62.5%. The difference between PY-BASE and PY-SBP-CORE is small: their overall win rates differ by 0.4 percentage points, and their 95% confidence intervals when playing second, [55.1%, 57.9%] and [55.0%, 57.7%], overlap almost entirely. The progression is therefore not monotonic across every position-conditioned result.

PY-SBP-STRATEGIES, which receives both the SBP Taki implementation and example strategy b-threads, records the highest overall win rate and the highest win rate in both turn positions. Its overall win rate is 3.1 percentage points higher than that of PY-BASE and 2.7 percentage points higher than that of PY-SBP-CORE.

The directly generated SBP-NATIVE strategy is evaluated separately because it changes the target artifact rather than extending the Python context progression. As shown in Table 3, it wins 64.9% of games against the same SBP-RANDOM baseline, including 67.4% when starting and 62.4% when playing second. Thus, casting the generated strategy itself as SBP b-threads also produces a strategy that outperforms the random baseline.

Together, these results support an affirmative answer to RQ3 for the evaluated configurations. Richer domain knowledge cast through SBP is associated with stronger generated strategies, with the highest observed Python performance obtained when example strategy b-threads are included. Because we eval-

uate one generated artifact per configuration, these results indicate an association and do not isolate a causal effect of any individual context component.

#### 5.4 Strategic Performance of SBP-NATIVE

We compare SBP-NATIVE directly with the generated Python strategies to evaluate whether a strategy generated as SBP b-threads remains competitive with strategies generated as conventional Python players. Table 4 reports the overall and position-conditioned win rates. In each comparison, P0 denotes the Python strategy and P1 denotes SBP-NATIVE.

Table 4: Head-to-head performance of the Python strategies and SBP-NATIVE over 10,000 games each.

| Configuration                    | Overall |       | Starting |       | Second |       |
|----------------------------------|---------|-------|----------|-------|--------|-------|
|                                  | P0      | P1    | P0       | P1    | P0     | P1    |
| PY-RANDOM vs. SBP-NATIVE         | 35.9%   | 64.1% | 38.9%    | 67.1% | 32.9%  | 61.1% |
| PY-BASE vs. SBP-NATIVE           | 47.7%   | 52.3% | 50.9%    | 55.6% | 44.4%  | 49.1% |
| PY-SBP-CORE vs. SBP-NATIVE       | 48.1%   | 51.9% | 52.4%    | 56.1% | 43.9%  | 47.6% |
| PY-SBP-STRATEGIES vs. SBP-NATIVE | 48.8%   | 51.2% | 53.5%    | 55.8% | 44.2%  | 46.5% |

SBP-NATIVE wins 64.1% of its games against PY-RANDOM. Against the three LLM-generated Python strategies, its overall win rate ranges from 51.2% to 52.3%. Its win rate is 2.3 percentage points above 50% against PY-BASE, narrowing to 1.2 percentage points against PY-SBP-STRATEGIES, which records the strongest result among the generated Python strategies in the preceding baseline comparison.

When playing second, SBP-NATIVE wins between 46.5% and 49.1% of its games against the generated Python strategies. In the same comparisons, the Python strategies win between 43.9% and 44.4% when playing second. Thus, SBP-NATIVE records the higher second-position win rate in each direct comparison, although all strategies perform better when starting.

We additionally compare SBP-NATIVE against three hand-authored SBP strategy refinements. Its overall win rate ranges from 52.3% to 52.5% in these matchups; the full results appear in the accompanying supplementary material.

Together, these results support an affirmative answer to RQ2 within the evaluated configurations: the directly generated SBP-NATIVE strategy achieves competitive performance relative to all generated Python strategies. The observed margins remain small, particularly against PY-SBP-STRATEGIES, and therefore do not support a general claim that SBP strategies are superior to Python ones.

## 6 Discussion

**Findings and Implications.** The Python progression provides evidence that casting domain knowledge through SBP can guide LLM-generated strategies.

The overall win rate increases as richer SBP-cast domain knowledge is provided, with the strongest Python result obtained by PY-SBP-STRATEGIES. The results suggest that example SBP strategy b-threads provide more useful guidance than exposing the rule-enforcement implementation alone, by illustrating how strategic knowledge can be expressed as modular behaviors.

The SBP-NATIVE configuration evaluates direct generation of strategy b-threads. In the evaluated simulations, the generated b-threads execute without observed runtime errors, deadlocks, or other execution failures, and they compose with an additional hand-authored b-thread. This suggests that SBP can provide a useful integration boundary for LLM-generated reactive code. The comparison between Python and SBP-native strategies indicates that the target representation affects the performance as well. Python strategies implement centralized heuristics, whereas SBP-native strategies influence event selection through modular b-threads, reducing the need to reproduce existing rule logic. Since SBP-NATIVE is only slightly ahead of PY-SBP-STRATEGIES, we interpret the result as evidence of competitiveness rather than broad superiority.

**Threats to Validity.** Several threats limit the generality of these findings. The evaluation uses one domain, one BPPy implementation, and a limited set of generated strategies. The results may also depend on the specific prompts, model, and generated artifacts. Moreover, the evaluation is simulation-based: the absence of observed runtime errors, deadlocks, or composition failures does not constitute a formal correctness guarantee. Finally, win rate captures strategic effectiveness in the implemented game, but does not fully measure code quality, maintainability, comprehensibility, or ease of human review.

## 7 Related Work

Large language models are increasingly used for software development tasks, including code generation, debugging, and program repair, with early work demonstrating both their ability to synthesize programs from natural-language descriptions and the remaining challenges in producing functionally correct code [8, 21]. Recent coding agents and repository-level benchmarks further highlight the importance of repository context, tool use, iterative revision, and cross-file reasoning for realistic software-engineering tasks [18, 22, 24]. This is especially relevant for reactive systems, where correctness depends on coordinated behavior over time rather than a single function in isolation. Recent work has therefore explored the use of LLMs for reactive-system development, including reactive controller generation, reactive synthesis, and modeling support [15, 20, 23].

Recent work studies the use of LLMs together with scenario-based programming frameworks. Yaacov et al. [30] propose using SBP as a general design structure for LLM-assisted development, where requirements are translated directly into separate b-threads and composed by the SBP execution mechanism. Harel et al. [13] present a broader methodology for integrating LLMs into scenario-based modeling, combining generation with manual inspection, simulation, and formal analysis. Berzack and Katz [7] present a related SBP-based workflow in a

Connect4 case study, where requirements are decomposed into scenarios, generated one b-thread at a time, and refined using inspection and, when applicable, verifier feedback.

Unlike prior work, which primarily focuses on generating scenario-based models or behavioral requirements from natural-language specifications, we study LLM-generated extensions to an existing reactive system. This shifts the challenge from generating standalone behaviors to composing new code with existing behavioral components, motivating our evaluation of executability, compositional integration, and large-scale simulation performance.

## 8 Conclusions and Future Work

Using the Taki card game as a case study, we find that LLM-generated SBP strategy b-threads integrate into the existing program without observed runtime errors, deadlocks, or draws in the evaluated simulations, while achieving competitive performance relative to LLM-generated Python strategies. In addition, richer domain knowledge cast through SBP is associated with stronger generated Python strategies in our simulations. These findings suggest that SBP can provide a useful structured representation for both communicating domain knowledge and generating composable extensions for reactive software.

Future work will evaluate the proposed methodology across additional reactive systems, SBP implementations, LLMs, and prompting strategies, as well as study software-engineering aspects beyond gameplay performance, including maintainability, comprehensibility, and human review effort. Another promising direction is to investigate richer forms of LLM-assisted SBP, such as composing multiple generated b-threads or iteratively refining them based on simulation or formal-analysis feedback.

## References

1. Anthropic: Claude sonnet 4.6 system card. <https://www-cdn.anthropic.com/78073f739564e986ff3e28522761a7a0b4484f84.pdf> (Feb 2026), accessed: 2026-06-28
2. Ashrov, A., Yaacov, T., Katz, G.: BP-Taki: A behavioral programming implementation of the taki card game. <https://github.com/adielashrov/bp-taki> (2026)
3. Ashrov, A., Yaacov, T., Katz, G.: BP-Taki: LLM-Assisted Strategy Generation using Behavioral Programming. <https://github.com/adielashrov/taki-llm-experiments> (2026), accessed: 2026
4. Bar-Sinai, M., Elyasaf, A., Sadon, A., Weiss, G.: A scenario based on-board software and testing environment for satellites. In: The 59th Israel Annual Conference on Aerospace Sciences (IACAS), 2019 (2019)
5. Bar-Sinai, M., Elyasaf, A., Weiss, G., Weiss, Y.: Provengo: A Tool Suite for Scenario Driven Model-Based Testing. In: Automated Software Engineering (ASE). pp. 2062–2065 (2023)
6. Bar-Sinai, M., Weiss, G., Shmuel, R.: BPjs: an extensible, open infrastructure for behavioral programming research. In: Proc. MODELS (2018)
7. Berzack, A., Katz, G.: On integrating large language models and scenario-based programming for improving software reliability. arXiv:2509.09194 (2025)

8. Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
9. Damm, W., Harel, D.: LSCs: Breathing life into message sequence charts. *Formal methods in system design* **19**(1), 45–80 (2001)
10. Elyasaf, A., Marron, A., Sturm, A., Weiss, G.: A Context-Based Behavioral Language for IoT. In: *MODELS Workshops*. pp. 485–494 (2018)
11. Gordon, M., Marron, A., Meerbaum-Salant, O.: Spaghetti for the Main Course? Observations on the Naturalness of Scenario-Based Programming. In: *Conf. on Innovation and Technology in Computer Science Education (ITICSE)* (2012)
12. Harel, D., Katz, G.: Scaling-Up Behavioral Programming: Steps from Basic Principles to Application Architectures. In: *Proc. AGERE!* pp. 95–108 (Oct 2014)
13. Harel, D., Katz, G., Marron, A., Szekely, S.: Enhancing scenario-based modeling using large language models. In: *Proc. Modelsward*. pp. 43–68. Springer (2025)
14. Harel, D., Marelly, R.: Come, let’s play: scenario-based programming using LSCs and the play-engine, vol. 1. Springer (2003)
15. Harel, D., Marron, A.: Preparing for super-reactivity: Early fault-detection in the development of exceedingly complex reactive systems. arXiv:2410.02627 (2024)
16. Harel, D., Marron, A., Weiss, G.: Programming Coordinated Behavior in Java. In: D’Hondt, T. (ed.) *ECOOP 2010 – Object-Oriented Programming* (2010)
17. Harel, D., Marron, A., Weiss, G.: Behavioral programming. *CACM* (2012)
18. Jimenez, C., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: *Proc. ICLR* (2024)
19. Katz, G., Marron, A., Sadon, A., Weiss, G.: On-the-fly construction of composite events in scenariobased modeling using constraint solvers. *Model-Driven Engineering and Software Development, MODELSWARD 2019* pp. 141–154 (2019)
20. Khan, M.T., Waheed, A.: Foundation model driven robotics: A comprehensive review. arXiv preprint arXiv:2507.10087 (2025)
21. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. arXiv:2305.01210 (2023)
22. Liu, T., Xu, C., McAuley, J.: RepoBench: Benchmarking repository-level code auto-completion systems. In: *Proc. ICLR* (2024)
23. Murphy, W., Holzer, N., Qiao, F., Cui, L., Rothkopf, R., Koenig, N., Santolucito, M.: Combining llm code generation with formal specifications and reactive program synthesis. arXiv preprint arXiv:2410.19736 (2024)
24. OpenAI: Introducing codex. <https://openai.com/index/introducing-codex/> (May 2025), accessed: 2026-06-28
25. OpenAI: Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/> (2026)
26. Shafir Games: TAKI Video Tutorial. <https://youtu.be/5-pFhiwwAME>
27. Shafir Games: Taki card game. <https://www.takigame.com/> (2026)
28. Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al.: Emergent abilities of large language models. arXiv preprint arXiv:2206.07682 (2022)
29. Yaacov, T.: BPy: Behavioral programming in Python. *SoftwareX* (2023)
30. Yaacov, T., Elyasaf, A., Weiss, G.: Boosting llm-based software generation by aligning code with requirements. In: *Proc. REW*. pp. 301–305. IEEE (2024)